# === UNIT 6: STRUCTURES, UNIONS, PREPROCESSOR & FILE HANDLING ===

## >> 6.1 Structures and Unions: Definition and Memory Alignment

Structures group heterogeneous variables under a single user-defined composite data type.
* struct Definition: struct Student { int id; char name[30]; float gpa; };. Members accessed using dot operator (s.id) for instances, or arrow operator (ptr->id) for structure pointers.
* Structure Padding and Memory Alignment: Compilers align structure members on natural byte boundaries (e.g., 4-byte integers align on addresses divisible by 4) to optimize CPU memory bus transactions. This inserts invisible padding bytes, causing sizeof(struct) to exceed algebraic sum of its members.
* unions: Syntactically identical to structures, but all members SHARE THE SAME MEMORY LOCATION. Size of union equals size of its largest member. Modifying one member overwrites all other members; used for type punning and hardware register maps.
* typedef: Creates descriptive aliases for existing types: typedef struct Student Student_t;.

## >> 6.2 Preprocessor Directives and Macros

* Macro Substitution: #define PI 3.14159. Function-like macros: #define SQUARE(x) ((x) * (x)) (parentheses essential to avoid operator precedence errors).
* Stringizing (#) and Token-Pasting (##) Operators:
- # operator converts macro parameter into quoted string: #define PRINT_VAR(x) printf(#x " = %d\n", x).
- ## operator concatenates two tokens into single identifier: #define GLUE(a, b) a##b.
* Conditional Compilation: #ifdef, #ifndef, #endif. Header Guards: Prevents multiple header file inclusion:
#ifndef HEADER_H \n #define HEADER_H \n /* declarations */ \n #endif.

## >> 6.3 File Management and Persistent Storage in C

* Stream Concept: In C, a file is an abstract byte stream accessed through a file pointer: FILE *fp;.
* File Opening Modes (fopen):
- "r": Open text file for reading (fails if file does not exist).
- "w": Open text file for writing (creates new file or truncates existing file to 0 bytes).
- "a": Open text file for appending (writes to end of file).

- "rb", "wb", "ab": Binary modes for non-text binary data.
* Formatted File I/O: fprintf(fp, "...", args), fscanf(fp, "...", &args).
* Character & String File I/O: fgetc(fp), fputc(ch, fp), fgets(buf, size, fp), fputs(str, fp).
* Direct Block Binary I/O: fwrite(const void *ptr, size_t size, size_t count, FILE *fp), fread(void *ptr, size_t size, size_t count, FILE *fp).
* File Positioning: fseek(fp, offset, whence) (whence: SEEK_SET, SEEK_CUR, SEEK_END), ftell(fp) (returns current byte position), rewind(fp). Always close files via fclose(fp).

---

**[PROGRAM 1: Structure Alignment and Padding Demonstration]**

---

Problem: Write an ANSI C program to demonstrate structure padding, compare struct vs. union memory footprints, and prove that union members share memory.

Source Code:

```
#include <stdio.h>
struct UnpaddedStruct {
char c; /* 1 byte + 3 bytes padding */
int i; /* 4 bytes */
short s; /* 2 bytes + 2 bytes padding */
}; /* Total = 12 bytes on 32/64-bit systems */
union SharedMemory {
int intVal;
float floatVal;
char str[4];
}; /* Total = 4 bytes (size of largest member) */
int main() {
printf("Size of UnpaddedStruct: %zu bytes (Sum of members = 1+4+2 = 7 bytes)\n",
sizeof(struct UnpaddedStruct));
printf("Size of SharedMemory Union: %zu bytes\n", sizeof(union SharedMemory));
union SharedMemory u;
u.intVal = 0x41424344;
printf("Union intVal = 0x%X\n", u.intVal);
printf("Union as chars: %c %c %c %c\n", u.str[0], u.str[1], u.str[2], u.str[3]);
return 0;
}
```

Output:

```
Size of UnpaddedStruct: 12 bytes (Sum of members = 1+4+2 = 7 bytes)
Size of SharedMemory Union: 4 bytes
Union intVal = 0x41424344
Union as chars: D C B A
```

---

**[PROGRAM 2: Student Database Management System via Binary Files]**

---

Problem: Write a program to: (1) Write an array of student records to a binary file using fwrite(), (2) Read and display records using fread().

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
int rollNo;
char name[30];
float gpa;
} Student_t;
int main() {
Student_t students[2] = {
{ 101, "Aarav Sharma", 8.95 },
{ 102, "Diya Verma", 9.40 }
};
FILE *fp = fopen("students.dat", "wb");
if (!fp) { printf("File open failed!\n"); return 1; }
fwrite(students, sizeof(Student_t), 2, fp);
fclose(fp);
printf("Records written to binary file successfully.\n");
Student_t readStudents[2];
fp = fopen("students.dat", "rb");
if (!fp) { printf("File read failed!\n"); return 1; }
fread(readStudents, sizeof(Student_t), 2, fp);
fclose(fp);
printf("Retrieved Records:\n");
for (int i = 0; i < 2; i++) {
printf(" Roll: %d | Name: %-15s | GPA: %.2f\n", readStudents[i].rollNo,
readStudents[i].name, readStudents[i].gpa);
}
return 0;
}
```

Output:

```
Records written to binary file successfully.
Retrieved Records:
Roll: 101 | Name: Aarav Sharma | GPA: 8.95
Roll: 102 | Name: Diya Verma | GPA: 9.40
```

---

**[PROGRAM 3: Preprocessor Stringizing and Token-Pasting Macro Pipeline]**

---

Problem: Implement logging macros in C demonstrating stringizing (#), token-pasting (##), and function-like macro expansions.

Source Code:

```
#include <stdio.h>
#define LOG_INT(var) printf("[DEBUG] Variable '" #var "' has value: %d\n", var)
#define CREATE_PAIR(type, name) type name##_first; type name##_second
#define SAFE_DIV(a, b) ((b) != 0 ? ((a) / (b)) : 0)
int main() {
int voltage = 230;
LOG_INT(voltage);
CREATE_PAIR(double, point);
point_first = 12.5;
point_second = 25.8;
printf("Point Coordinates: (%.1f, %.1f)\n", point_first, point_second);
int q1 = SAFE_DIV(100, 5);
int q2 = SAFE_DIV(100, 0);
printf("Safe Division: 100/5 = %d | 100/0 = %d\n", q1, q2);
return 0;
}
```

Output:

```
[DEBUG] Variable 'voltage' has value: 230
Point Coordinates: (12.5, 25.8)
Safe Division: 100/5 = 20 | 100/0 = 0
```

---

**[PROGRAM 4: Direct File Random Access via fseek and ftell]**

---

Problem: Write a program that writes alphabet 'A' through 'Z' to a file, then uses fseek() and ftell() to read characters at specific byte offsets randomly.

Source Code:

```
#include <stdio.h>
int main() {
FILE *fp = fopen("alphabet.txt", "w+");
if (!fp) return 1;
for (char c = 'A'; c <= 'Z'; c++) fputc(c, fp);
fseek(fp, 0, SEEK_END);
long fileSize = ftell(fp);
printf("Total file size: %ld bytes\n", fileSize);
fseek(fp, 0, SEEK_SET);
printf("Position 0: %c\n", fgetc(fp));
fseek(fp, 4, SEEK_SET);
printf("Position 4: %c\n", fgetc(fp));
fseek(fp, -5, SEEK_END);
printf("5 bytes from end: %c\n", fgetc(fp));
fclose(fp);
return 0;
}
```

Output: Total file size: 26 bytes | Position 0: A | Position 4: E | 5 bytes from end: V.

### >> 6.4 University Review & Examination Questions

* Q1: Contrast structures (struct) and unions (union) regarding memory allocation, member accessibility, and practical use cases.
Answer: (1) Memory Allocation: In a struct, every member has its own separate memory location. Total size is at least sum of member sizes plus compiler padding. In a union, all members share SAME memory location. Size is strictly equal to size of largest member. (2) Member Accessibility: In struct, all members can store distinct data concurrently. In union, only ONE member holds meaningful data at any given instant; writing to one member corrupts others. (3) Use Cases: Structures model entities with multiple simultaneous attributes (student records, coordinate points). Unions used when memory is scarce (embedded microcontrollers) or when an entity takes one of several mutually exclusive formats (variant data types, network packet headers).
* Q2: Explain the difference between text files and binary files in C, detailing fread() and fwrite() operations.
Answer: Text files store data as human-readable ASCII or UTF-8 characters where each line terminates with newline delimiter ('\n' on UNIX, '\r\n' on Windows). During reading/writing, runtime performs translation. Binary files store data as raw, unformatted byte sequences mirroring exact memory representations. Binary files require no translation, offer higher I/O speed, and prevent rounding errors. Block I/O functions: 'fwrite(buffer, size, count, fp)' writes 'count' elements of 'size' bytes from buffer to stream; 'fread(buffer, size, count, fp)' reads raw bytes directly into memory. Both return number of complete items successfully processed.

### >> 6.5 Key Takeaways and Revision Points

* Structures group diverse data; compilers add padding bytes to satisfy alignment requirements.
* Union members share a single memory block; size equals largest member.
* Preprocessor directives (#define, #ifdef) execute prior to compilation; header guards prevent multiple inclusions.
* File I/O uses stream pointers (FILE*); binary files use fread/fwrite; fseek enables random access.